

Lab Checkpoint 0: networking warmup

Due: Sunday, September 28, 11:59 p.m.

Welcome to CS144: Introduction to Computer Networking. In this warmup, you will set up an installation of GNU/Linux on your computer, learn how to perform some tasks over the Internet by hand, write a small program in C++ that fetches a Web page over the Internet, and implement (in memory) one of the key abstractions of networking: a reliable stream of bytes between a writer and a reader. We expect this warmup to take you between 2 and 6 hours to complete (future checkpoints will take more of your time). Three quick points about the lab assignment:

- It’s a good idea to read the whole document before diving in!
- Over the course of this 8-part lab assignment, you’ll be building up your own implementation of a significant portion of the Internet—a router, a network interface, and the TCP protocol (which transforms unreliable datagrams into a reliable byte stream). *Most weeks will build on work you have done previously*, i.e., you are building up your own implementation gradually over the course of the quarter, and you’ll continue to use your work in future weeks. This makes it hard to “skip” a checkpoint.
- If you don’t meet the CS144 prerequisites, please don’t take this class yet—our teaching staff’s resources are limited. And please use checkpoints 0 and 1 as a gauge: if you find yourself uncomfortable with the programming in the first two checkpoints, **please consider taking CS144 in a later year** after you’ve attained more comfort with this kind of programming (perhaps after taking CS 106L, embarking on a self-directed programming project, or otherwise building up your comfort and experience level).
- The lab documents aren’t “specifications”—meaning they’re not intended to be consumed in a one-way fashion. They’re written closer to the level of detail that a software engineer will get from a boss or client. We expect that you’ll benefit from attending the lab sessions and asking clarifying questions if you find something to be ambiguous and you think the answer matters. We’ll update the “lab FAQ” document on the [course website](#) in response to late questions that need clarification.

0 Collaboration Policy

The programming assignments must be your own work: You must write all the code you hand in for the programming assignments, except for the code that we give you as part of the assignment. Please do not copy-and-paste code from Stack Overflow, GitHub, ChatGPT, or other sources. If you base your own code on examples you find on the Web or elsewhere, cite the URL in a comment in your submitted source code.

Working with others: You may not show your code to anyone else, look at anyone else’s code for these assignments (whether human- or AI-authored), or look at solutions from previous years. You may discuss the assignments with other students, but do not copy anybody’s code. If you discuss an assignment with another student, please name them in a

实验检查点 0：网络热身

截止日期：9 月 28 日（星期日）晚上 11:59

欢迎来到 CS144：计算机网络导论。在本次热身练习中，你将在计算机上安装 GNU/Linux 系统，学习如何手动通过互联网执行一些任务，编写一个用 C++ 实现的、能通过互联网获取网页的小程序，并在内存中实现网络通信的核心抽象之一：在写入方和读取方之间建立可靠的字节流。我们预计完成本次热身练习需要 2 到 6 小时（后续的检查点将花费更多时间）。关于实验作业的三点快速说明：

- 建议在开始前通读整份文档！
- 在这项由 8 个部分组成的实验作业中，你将逐步构建互联网重要组成部分的自主实现—包括路由器、网络接口以及 TCP 协议（该协议能将不可靠的数据报转换为可靠的字节流）。大多数周次的任务都基于你之前完成的工作，也就是说，你将在整个学期中逐步构建自己的实现，并在后续周次中持续使用你的成果。这使得“跳过”某个检查点变得困难。
- 如果你不符合 CS144 的先修课程要求，请暂时不要选修这门课—我们的教学团队资源有限。同时，请将检查点 0 和 1 作为衡量标准：如果你在前两个检查点的编程任务中感到吃力，建议你在后续学年中，待自己对此类编程更加得心应手后再选修 CS144（或许可以先修读 CS 106L 课程、开展自主编程项目，或通过其他方式提升编程熟练度和经验水平）。
- 实验文档并非“规范说明”—也就是说，它们并非单向灌输的材料。其详细程度更接近软件工程师从老板或客户那里获得的任务说明。我们建议你参加实验课，如果发现某些内容存在歧义且你认为答案至关重要，请提出澄清问题。我们会根据后续需要澄清的提问，在课程网站上更新“实验常见问题解答”文档。

0 合作政策

编程作业必须由你独立完成：除课程提供的代码外，所有提交的编程作业代码必须由你亲自编写。请勿从 Stack Overflow、GitHub、ChatGPT 或其他来源复制粘贴代码。若你的代码基于网络或其他渠道的示例，请在提交的源代码注释中注明相关 URL。

合作规范：不得向他人展示你的代码，不得查看他人（无论是人类还是 AI 编写）的作业代码，也不得查阅往年答案。你可以与其他同学讨论作业内容，但严禁复制任何人的代码。若与同学讨论过作业，请在提交文件中注明对方姓名。

comment in your submitted source code. Please refer to the course administrative handout for more details, and ask on EdStem if anything is unclear. Services like GitHub Copilot or ChatGPT should be considered to be equivalent to “a student that took CS144 in a prior year.”

EdStem: Please feel free to ask questions on EdStem, but please don’t post any source code.

1 Set up GNU/Linux on your computer

CS144’s assignments require the GNU/Linux operating system and a recent C++ compiler that supports the C++ 2023 standard. Please choose one of these three options:

1. **Recommended:** Install the CS144 VirtualBox virtual-machine image (instructions at <https://stanford.edu/class/cs144/vm-howto/vm-howto-image.html>).
2. Use a Google Cloud virtual machine (instructions at <https://stanford.edu/class/cs144/vm-howto>). We have applied for \$50 coupons from Google and will distribute them if/when they arrive.
3. Run Ubuntu version 25.04.
4. Use another GNU/Linux distribution “at your own risk,” but be aware that you may hit roadblocks along the way and will need to be comfortable debugging them. Your code will be tested on Ubuntu 25.04 LTS with g++ 14.2.0 and must compile and run properly under those conditions.
5. If you have a 2020–24 MacBook (with the ARM64 M-series chips), VirtualBox will not successfully run. Instead, please install the UTM virtual machine software and our ARM64 virtual machine image from <https://stanford.edu/class/cs144/vm-howto/>.

2 Install the required packages

These commands will install the required Ubuntu (Debian) packages to run the course software:

```
sudo apt update && sudo apt install git cmake gdb build-essential clang \
    clang-tidy clang-format gcc-doc pkg-config glibc-doc tcpdump tshark
```

3 Networking by hand

Let’s get started with using the network. You are going to do two tasks by hand: retrieving a Web page (just like a Web browser) and sending an email message (like an email client). Both of these tasks rely on a networking abstraction called a *reliable bidirectional byte stream*: you’ll type a sequence of bytes into the terminal, and the same sequence of bytes will eventually

在你提交的源代码中注明注释。更多详情请参考课程管理手册，如有任何疑问请在 EdStem 上提问。GitHub Copilot 或 ChatGPT 等工具应被视为“往年修读过 CS144 课程的学生”同等对待。

EdStem：欢迎在 EdStem 上提问，但请勿发布任何源代码。

1 在你的计算机上安装 GNU/Linux 系统

CS144 的作业需要 GNU/Linux 操作系统以及支持 C++ 2023 标准的最新 C++编译器。请从以下三种方案中选择一种：

1. 推荐：安装 CS144 VirtualBox 虚拟机镜像（操作指南详见 <https://stanford.edu/class/cs144/vm-howto/vm-howto-image.html>）。
2. 2. 使用谷歌云虚拟机（操作指南详见 <https://stanford.edu/class/cs144/vm-howto>）。我们已向谷歌申请了 50 美元优惠券，待发放后将进行分配。
3. 运行 Ubuntu 25.04 版本。
4. 使用其他 GNU/Linux 发行版需“自行承担风险”，但请注意过程中可能遇到障碍，且需具备调试能力。你的代码将在 Ubuntu 25.04 LTS 系统及 g++ 14.2.0 环境下进行测试，必须确保在此条件下编译与运行正常。
5. 如果你使用的是 2020–24 款 MacBook（搭载 ARM64 M 系列芯片），VirtualBox 将无法正常运行。请改用 UTM 虚拟机软件，并从 <https://stanford.edu/class/cs144/vm-howto/> 下载我们的 ARM64 虚拟机镜像。

2 安装所需软件包

以下命令将安装运行课程软件所需的 Ubuntu (Debian) 软件包：

```
sudo apt update && sudo apt install git cmake gdb build-essential clang \
    clang-tidy clang-format gcc-doc pkg-config glibc-doc tcpdump tshark
```

3 手动网络操作

让我们开始使用网络。你将手动完成两项任务：检索一个网页（就像网页浏览器那样）和发送一封电子邮件（就像电子邮件客户端那样）。这两项任务都依赖于一种称为可靠双向字节流的网络抽象：你将在终端中输入一串字节，而同样的字节串最终会

be delivered, in the same order, to a program running on another computer (a server). The server responds with its own sequence of bytes, delivered back to your terminal.

3.1 Fetch a Web page

1. In a Web browser, visit <http://cs144.keithw.org/hello> and observe the result.
2. Now, you'll do the same thing the browser does, by hand.

(a) **On your VM** (or on your own computer—e.g. the Terminal program on macOS), run `telnet cs144.keithw.org http`. This tells the `telnet` program to open a reliable byte stream between your computer and another computer (named `cs144.keithw.org`), and with a particular *service* running on that computer: the “`http`” service, for the Hyper-Text Transfer Protocol, used by the World Wide Web.¹

If your computer has been set up properly and is on the Internet, you will see:

```
user@computer:~$ telnet cs144.keithw.org http
Trying 104.196.238.229...
Connected to cs144.keithw.org.
Escape character is '^['.
```

If you need to quit, hold down `ctrl` and press `]`, and then type `close`.

(b) Type `GET /hello HTTP/1.1`. This tells the server the *path* part of the URL. (The part starting with the third slash.)

(c) Type `Host: cs144.keithw.org`. This tells the server the *host* part of the URL. (The part between `http://` and the third slash.)

(d) Type `Connection: close`. This tells the server that you are finished making requests, and it should close the connection as soon as it finishes replying.

(e) Hit the Enter key one more time: . This sends an empty line and tells the server that you are done with your HTTP request.

(f) If all went well, you will see the same response that your browser saw, preceded by HTTP *headers* that tell the browser how to interpret the response.

3. **Assignment:** Now that you know how to fetch a Web page by hand, show us you can! Use the above technique to fetch the URL <http://cs144.keithw.org/lab0/sunetid>, replacing *sunetid* with your own primary SUNet ID. You will receive a secret code in the `X-Your-Code-Is:` header. Save your SUNet ID and the code for inclusion in your writeup.

¹The computer’s name has a numerical equivalent (104.196.238.229, an *Internet Protocol v4 address*), and so does the service’s name (80, a *TCP port number*). We’ll talk more about these later.

这两项任务都依赖于一种称为可靠双向字节流的网络抽象：你将在终端中输入一串字节，而同样的字节串最终会CS144：计算机网络导论

按相同顺序传输到另一台计算机（服务器）上运行的程序。服务器会返回自己的字节序列，并传回你的终端。

3.1 获取一个网页

1. 在网页浏览器中访问 `http://cs144.keithw.org/hello` 并观察结果。
2. 现在，你将手动完成浏览器所做的相同操作。（a）在你的虚拟机（或你自己的电脑上一例如 macOS 上的终端程序）中，

运行 `telnet cs144.keithw.org http` 命令。这会指示 `telnet` 程序在你的计算机与另一台名为 `cs144.keithw.org` 的计算机之间建立一个可靠的字节流，并与该计算机上运行的特定服务进行通信：即“`http`”服务，用于万维网中的超文本传输协议。

如果你的计算机已正确配置并连接至互联网，你将看到：

```
user@computer:~$ telnet cs144.keithw.org http
Trying 104.196.238.229...
已连接到 cs144.keithw.org。
转义字符为 '^['。
```

如需退出，请按住 `Ctrl` 键并按下 `]` 键，然后输入 `close`。

(b) 输入 `GET /hello HTTP/1.1`。这将告知服务器 URL 的路径部分。（从第三个斜杠开始的部分。）

(c) 输入 `Host: cs144.keithw.org`。这告诉服务器 URL 的主机部分。（即 `http://` 与第三个斜杠之间的部分。）

(d) 输入 `Connection: close`。这告诉服务器你已经完成请求，它应在回复完毕后立即关闭连接。

(e) 再按一次回车键：。这会发送一个空行，并告诉服务器你的 HTTP 请求已结束。

(f) 如果一切顺利，你将看到与浏览器相同的响应，前面还带有 HTTP 头部信息，这些头部信息告诉浏览器如何解释该响应。

3. 任务：既然你已经知道如何手动获取网页，那就展示一下你的能力吧！使用上述技术获取 URL `http://cs144.keithw.org/lab0/sunetid`，并将 `sunetid` 替换为你自己的主 SUNet ID。你将在 `X-Your-Code-Is:` 头部中收到一个秘密代码。请保存你的 SUNet ID 和该代码，以便在报告中包含。

计算机名称有一个数字等效形式（104.196.238.229，一个互联网协议第四版地址），服务名称也有一个数字等效形式（80，一个 TCP 端口号）。我们稍后会详细讨论这些内容。

3.2 Send yourself an email

Now that you know how to fetch a Web page, it’s time to send an email message, again using a reliable byte stream to a service running on another computer.

1. SSH to `sunetid@cardinal.stanford.edu` (to make sure you are on Stanford’s network), then run `telnet 67.231.149.169 smtp`.² The “smtp” service refers to the Simple Mail Transfer Protocol, used to send email messages. If all goes well, you will see:
2. First step: identify your computer to the email server. Type `HELO mycomputer.stanford.edu`. Wait to see something like “250 ... Hello cardinal3.stanford.edu [171.67.24.75], pleased to meet you”.
3. Next step: who is sending the email? Type `MAIL FROM: sunetid@stanford.edu`. Replace `sunetid` with your SUNet ID.³ If all goes well, you will see “250 2.1.0 Sender ok”.
4. Next: who is the recipient? For starters, try sending an email message to yourself. Type `RCPT TO: sunetid@stanford.edu`. Replace `sunetid` with your own SUNet ID. If all goes well, you will see “250 2.1.5 Recipient ok.”
5. It’s time to upload the email message itself. Type `DATA` to tell the server you’re ready to start. If all goes well, you will see “354 End data with <CR><LF>.<CR><LF>”.
6. Now you are typing an email message to yourself. First, start by typing the *headers* that you will see in your email client. Leave a blank line at the end of the headers.

```
354 End data with <CR><LF>.<CR><LF>
From: sunetid@stanford.edu
To: sunetid@stanford.edu
Subject: Hello from CS144 Lab 0!
```

²These instructions might also work from outside Stanford’s network, but we can’t guarantee it.

³Yes, it’s possible to give a phony “from” address. Electronic mail is a bit like real mail from the postal service, in that the accuracy of the return address is (mostly) on the honor system. You can write anything you like as the return address on a postcard, and the same is largely true of email. Please do not abuse this—seriously. With engineering knowledge comes responsibility! Sending email with a phony “from” address is commonly done by spammers and criminals so they can pretend to be somebody else. It’s fun to play around with this and pretend to be `santaclaus@northpole.gov`, but **make sure you don’t deceive any recipient**. And: even if the recipient is in on the joke, **do not send email pretending to be any Stanford employee** (otherwise you may set off the university’s IT security alerts).

3.2 给自己发送一封电子邮件

既然你已经知道如何获取网页，现在是时候发送一封电子邮件了，同样使用可靠的字节流连接到另一台计算机上运行的服务。

1. 通过 SSH 登录到 `sunetid@cardinal.stanford.edu` (确保你在斯坦福的网络中)，然后运行 `telnet 67.231.149.169 smtp`。“smtp”服务指的是简单邮件传输协议。邮件传输协议，用于发送电子邮件。如果一切顺利，你将看到：
2. 第一步：向邮件服务器标识你的计算机。输入 `HELO mycomputer.stanford.edu`。等待看到类似“250 ... Hello cardinal3.stanford.edu [171.67.24.75], pleased to meet you”的响应。
3. 下一步：谁在发送邮件？输入 `MAIL FROM: sunetid@stanford.edu`。将 `sunetid` 替换为你的 SUNet ID。如果一切顺利，你会看到“250 2.1.0 Sender ok”。
4. 下一步：收件人是谁？首先，尝试给自己发送一封邮件。输入 `RCPT TO: sunetid@stanford.edu`。将 `sunetid` 替换为你自己的 SUNet ID。如果一切顺利，您将看到“250 2.1.5 Recipient ok.”。
5. 现在可以上传邮件正文了。输入 `DATA` 告知服务器您已准备好开始。如果一切顺利，您将看到“354 End data with .”。
6. 现在您正在给自己撰写一封邮件。首先，输入您将在邮件客户端中看到的邮件头。在邮件头末尾留一个空行。

```
354 以.结束数据。发件人:
sunetid@stanford.edu 收件人:
sunetid@stanford.edu 主题: 来自 CS144 实验 0 的问候!
```

²这些说明在斯坦福网络之外也可能有效，但我们无法保证。

是的，可以伪造发件人地址。电子邮件与邮政服务的实体邮件有些相似，因为回信地址的准确性（主要）依赖于诚信原则。你可以在明信片上随意填写回信地址，电子邮件也大致如此。请勿滥用此功能—这是严肃的警告。掌握工程知识意味着要承担责任！发送伪造发件人地址的电子邮件通常是垃圾邮件发送者和犯罪分子所为，以便冒充他人。虽然用这个功能假装成 `santaclaus@northpole.gov` 玩一玩很有趣，但请确保不要欺骗任何收件人。此外：即使收件人知道这是个玩笑，也不要发送冒充斯坦福员工的邮件（否则可能触发大学的信息安全警报）。

7. Type the *body* of the email message—anything you like. When finished, end with a dot on a line by itself: `. ↵`. Expect to see something like: “250 2.0.0 33h24dpdsr-1 Message accepted for delivery”.
8. Type `QUIT ↵` to end the conversation with the email server. Check your inbox and spam folder to make sure you got the email.

3.3 Listening and connecting

You’ve seen what you can do with `telnet`: a **client** program that makes outgoing connections to programs running on other computers. Now it’s time to experiment with being a simple **server**: the kind of program that waits around for clients to connect to it.

1. In one terminal window, run `netcat -v -l -p 9090` on your VM. You should see:

```
user@computer:~$ netcat -v -l -p 9090
Listening on [0.0.0.0] (family 0, port 9090)
```
2. Leave `netcat` running. In another terminal window, run `telnet localhost 9090` (also on your VM).
3. If all goes well, the `netcat` will have printed something like “Connection from localhost 53500 received!”.
4. Now try typing in either terminal window—the `netcat` (server) or the `telnet` (client). Notice that anything you type in one window appears in the other, and vice versa. You’ll have to hit `↵` for bytes to be transfered.
5. In the `netcat` window, quit the program by typing `ctrl-C`. Notice that the `telnet` program immediately quits as well.

4 Writing a network program using an OS stream socket

In the next part of this warmup lab, you will write a short program that fetches a Web page over the Internet. You will make use of a feature provided by the Linux kernel, and by most other operating systems: the ability to create a *reliable bidirectional byte stream* between two programs, one running on your computer, and the other on a different computer across the Internet (e.g., a Web server such as Apache or nginx, or the `netcat` program).

This feature is known as a *stream socket*. To your program and to the Web server, the socket looks like an ordinary file descriptor (similar to a file on disk, or to the `stdin` or `stdout`

7. 输入邮件正文—内容随意。完成后，在单独一行输入句点结束：`. .` 预期会看到类似这样的响应：`"250 2.0.0 33h24dpdsr-1 Message accepted for delivery"`。
8. 输入 `QUIT 结束`与邮件服务器的对话。检查收件箱和垃圾邮件文件夹，确保已收到邮件。

3.3 监听与连接

你已经见识过 `telnet` 的用途：一个客户端程序，用于向外连接其他计算机上运行的程序。现在，是时候尝试扮演一个简单的服务器角色了：这种程序会持续等待客户端连接。

1. 在一个终端窗口中，在您的虚拟机上运行 ``netcat -v -l -p 9090``。您应该会看到：

```
`user@computer:~$ netcat -v -l -p 9090`
```

正在监听 [0.0.0.0]（地址族 0，端口 9090）
2. 保持 `netcat` 运行。在另一个终端窗口中，运行 ``telnet localhost 9090``（同样在您的虚拟机上）。
3. 如果一切顺利，`netcat` 将打印出类似“来自 localhost 53500 的连接已接收！”的信息。
4. 现在尝试在两个终端窗口（`netcat` 服务器端或 `telnet` 客户端）中输入内容。注意，你在一个窗口中输入的任何内容都会出现在另一个窗口中，反之亦然。你需要按下回车键才能传输字节数据。
5. 在 `netcat` 窗口中，通过按 `Ctrl+C` 退出程序。注意，`telnet` 程序也会立即退出。

4 使用操作系统流式套接字编写网络程序

在本次热身实验的下一部分中，你将编写一个简短的程序，用于通过互联网获取网页。你将利用 Linux 内核以及大多数其他操作系统提供的一项功能：即在两个程序之间创建可靠的、双向的字节流，其中一个程序运行在你的计算机上，另一个则运行在互联网上另一台不同的计算机上（例如，Apache 或 nginx 等 Web 服务器，或 `netcat` 程序）。这项功能被称为流套接字。对于你的程序和 Web 服务器而言，这个套接字看起来就像一个普通的文件描述符（类似于磁盘上的文件，或标准输入/标准输出）。

I/O streams). When two stream sockets are *connected*, any bytes written to one socket will eventually come out in the same order from the other socket on the other computer.

In reality, however, the Internet doesn’t provide a service of reliable byte-streams. Instead, the only thing the Internet really does is to give its “best effort” to deliver short pieces of data, called *Internet datagrams*, to their destination. Each datagram contains some metadata (headers) that specifies things like the source and destination addresses—what computer it came from, and what computer it’s headed towards—as well as some *payload* data (up to about 1,500 bytes) to be delivered to the destination computer.

Although the network tries to deliver every datagram, in practice datagrams can be (1) lost, (2) delivered out of order, (3) delivered with the contents altered, or even (4) duplicated and delivered more than once. It’s normally the job of the operating systems on either end of the connection to turn “best-effort datagrams” (the abstraction the Internet provides) into “reliable byte streams” (the abstraction that applications usually want).

The two computers have to cooperate to make sure that each byte in the stream eventually gets delivered, in its proper place in line, to the stream socket on the other side. They also have to tell each other how much data they are prepared to accept from the other computer, and make sure not to send more than the other side is willing to accept. All this is done using an agreed-upon scheme that was set down in 1981, called the Transmission Control Protocol, or TCP.

In this lab, you will simply use the operating system’s pre-existing support for the Transmission Control Protocol. You’ll write a program called “**webget**” that creates a TCP stream socket, connects to a Web server, and fetches a page—much as you did earlier in this lab. In future labs, you’ll implement the other side of this abstraction, by implementing the Transmission Control Protocol yourself to create a reliable byte-stream out of not-so-reliable datagrams.

4.1 Let’s get started—setting up the repository on your VM and on GitHub

1. The lab assignments will use a starter codebase called “Minnow.” **On your VM**, run `git clone https://github.com/cs144/minnow` to fetch the source code for the lab.
2. Enter the minnow directory by typing: `cd minnow`
3. In a Web browser, you’ll make a repository within your own GitHub account to hold your solutions to the lab assignment.
 - (a) If you don’t already have a GitHub account, please make one at <https://github.com>.
 - (b) Navigate to <https://github.com/new> to create a new repository.
 - (c) Name the repository “minnow” within your GitHub account.
 - (d) *Make sure to set the repository to “Private” so your solutions are not public.*

I/O 流)。当两个流式套接字建立连接后，写入一个套接字的任何字节，最终都会以相同的顺序从另一台计算机的另一个套接字中输出。

然而，实际上互联网并不提供可靠的字节流服务。相反，互联网真正做的只是“尽力”将称为互联网数据报的短数据片段传送到其目的地。每个数据报都包含一些元数据（头部），用于指定源地址和目标地址等信息——即数据来自哪台计算机、发往哪台计算机——以及一些要传送到目标计算机的有效载荷数据（最多约 1500 字节）。

尽管网络尽力传输每个数据报，但在实际中，数据报可能发生以下情况：（1）丢失，（2）乱序到达，（3）内容被篡改，甚至（4）被重复传输多次。通常，连接两端的操作系统需要将“尽力而为的数据报”（互联网提供的抽象）转化为套接字，使其看起来像普通的文件描述符（类似于磁盘上的文件），从而形成“可靠的字节流”（应用程序通常需要的抽象）。

两台计算机必须协作，确保流中的每个字节最终都能按正确顺序传递到另一端的流式套接字。它们还需要相互告知各自准备接收的数据量，并确保不会发送超过对方愿意接收的数据量。这一切都通过 1981 年制定的统一方案——传输控制协议（TCP）来实现。

在本实验中，你将直接使用操作系统对传输控制协议的现有支持。你需要编写一个名为“webget”的程序，该程序会创建一个 TCP 流套接字，连接到 Web 服务器并获取网页——这与你在本实验之前所做的操作非常相似。在后续实验中，你将通过自行实现传输控制协议，从不可靠的数据报中构建可靠的字节流，从而实现这一抽象的另一端。

4.1 让我们开始吧—在你的虚拟机和 GitHub 上配置仓库

1. 实验任务将使用名为“Minnow”的初始代码库。在你的虚拟机上运行 `git clone https://github.com/cs144/minnow` 来获取实验的源代码。
2. 输入 `cd minnow` 进入 minnow 目录。
3. 在网页浏览器中，你需要在个人 GitHub 账户内创建一个仓库，用于存放实验作业的解答。
 - (a) 如果尚未拥有 GitHub 账户，请前往 <https://github.com> 注册一个。
 - (b) 访问 <https://github.com/new> 创建一个新仓库。
 - (c) 在你的 GitHub 账户中将仓库命名为“minnow”。
 - (d) 确保将仓库设置为“私有”，这样你的解决方案就不会公开。

- (e) Click “Create Repository”.

(f) On the next screen, click “Invite collaborators”, then “Add people”.

(g) Add “cs144-grader” as a collaborator (this will let us see and grade your code, while keeping it private).
4. Back on your VM, register the GitHub repository as a target by running the command:
`git remote add github https://github.com/username/minnow` (replacing “username” with your actual GitHub username). This creates an association between your local copy of the lab assignment (on your VM) and your copy on GitHub (which you’ll use to back up your local copy and be graded).
5. Run `git push github` to send the starter code to your GitHub repository. If all goes well, you will see a few lines of text printed, ending in: `* [new branch] main -> main`. If you see an error message, double-check that you have executed the above steps correctly. This command uploads *your* code to your private copy of the repository on GitHub and lets us grade your submissions.

4.2 Compiling the starter code

1. Still in the “minnow” directory, create a directory to compile the lab software: `cmake -S . -B build`
2. Compile the source code: `cmake --build build`
3. Using your favorite text editor (many students prefer VS Code editing files over SSH, but you can use whatever you want): open and start editing the `writeups/check0.md` file. This is the template for your lab checkpoint writeup and will be included in your submission.

4.3 Modern C++: mostly safe but still fast and low-level

CS144 is a programming-heavy class. The lab assignment is done in a contemporary C++ style that uses recent (2023) features to program as safely as possible. This might be different from how you have been asked to write C++ in the past. For references to this style, please see the C++ Core Guidelines (<http://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines>).

The basic idea is to make sure that every object is designed to have the smallest possible public interface, has a lot of internal safety checks and is hard to use improperly, and knows how to clean up after itself. We want to avoid “paired” operations (e.g. `malloc/free`, or `new/delete`), where it might be possible for the second half of the pair not to happen (e.g., if a function returns early or throws an exception). Instead, operations happen in the constructor to an object, and the opposite operation happens in the destructor. This style is called “Resource acquisition is initialization,” or RAII.

- (e) 点击“创建仓库”。

(f) 在下一个界面中，点击“邀请协作者”，然后点击“添加人员”。

(g) 将“cs144-grader”添加为协作者（这样我们就能查看并评分你的代码，同时保持其私密性）。
4. 回到你的虚拟机，通过运行以下命令将 GitHub 仓库注册为目标仓库：
`git remote add github https://github.com/username/minnow`
(将“username”替换为你实际的 GitHub 用户名)。这会在你本地（虚拟机中）的实验作业副本与 GitHub 上的副本（用于备份本地代码并接受评分）之间建立关联。
5. 运行 `git push github` 将初始代码推送到你的 GitHub 仓库。如果一切顺利，你会看到几行文本输出，最后一行是: `* [new branch] main -> main`。如果看到错误信息，请仔细检查上述步骤是否执行正确。此命令会将你的代码上传到 GitHub 上你的私有仓库副本中，以便我们评分你的提交。

4.2 编译入门代码

1. 仍在 “minnow” 目录下，创建一个目录来编译实验软件: `cmake -S . -B build`
2. 编译源代码: `cmake --build build`
3. 使用你喜欢的文本编辑器（许多学生更喜欢通过 SSH 使用 VS Code 编辑文件，但你可以使用任何你想要的编辑器）：打开并开始编辑 `writeups/check0.md` 文件。这是你的实验检查点报告的模板，将包含在你的提交中。

4.3 现代 C++：基本安全，但仍保持高速与底层能力

CS144 是一门编程密集型课程。实验作业采用当代 C++ 风格，利用最新（2023 年）特性尽可能安全地编写程序。这可能与你以往被要求编写 C++ 的方式有所不同。关于这种风格的参考资料，请参阅 C++ 核心指南 (<http://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines>)。

基本理念是确保每个对象都设计为拥有尽可能小的公共接口，具备大量内部安全检查，难以被不当使用，并且知道如何自行清理。我们希望避免“成对”操作（例如 `malloc/free` 或 `new/delete`），因为这类操作中后半部分可能无法执行（例如函数提前返回或抛出异常时）。取而代之的是，操作在对象的构造函数中执行，而反向操作则在析构函数中完成。这种风格被称为“资源获取即初始化”（RAII）。

In particular, we would like you to:

- Use the language documentation at <https://en.cppreference.com> as a resource. (We'd recommend you avoid `cplusplus.com` which is more likely to be out-of-date.)
- Never use `malloc()` or `free()`.
- Never use `new` or `delete`.
- Essentially never use raw pointers (*), and use “smart” pointers (`unique_ptr` or `shared_ptr`) only when necessary. (You will not need to use these in CS144.)
- Avoid templates, threads, locks, and virtual functions. (You will not need to use these in CS144.)
- Avoid C-style strings (`char *str`) or string functions (`strlen()`, `strcpy()`). These are pretty error-prone. Use a `std::string` instead.
- Never use C-style casts (e.g., `(FILE *)x`). Use a C++ `static_cast` if you have to (you generally will not need this in CS144).
- Prefer passing function arguments by `const` reference (e.g.: `const Address & address`).
- Make every variable `const` unless it needs to be mutated.
- Make every method `const` unless it needs to mutate the object.
- Avoid global variables, and give every variable the smallest scope possible.
- Before handing in an assignment, run `cmake --build build --target tidy` for suggestions on how to improve the code related to C++ programming practices, and `cmake --build build --target format` to format the code consistently.

On using Git: The labs are distributed as Git (version control) repositories—a way of documenting changes, checkpointing versions to help with debugging, and tracking the provenance of source code. **Please make frequent small commits as you work, and use commit messages that identify what changed and why.** The Platonic ideal is that each commit should compile and should move steadily towards more and more tests passing. Making small “semantic” commits helps with debugging (it's much easier to debug if each commit compiles and the message describes one clear thing that the commit does) and protects you against claims of cheating by documenting your steady progress over time—and it's a useful skill that will help in any career that includes software development. The graders will be reading your commit messages to understand how you developed your solutions to the labs. If you haven't learned how to use Git, please do ask for help at the CS144 office hours or consult a tutorial (e.g., <https://guides.github.com/introduction/git-handbook>). Finally, while we ask you to back your code up and submit your code to us by using a **private** repository on GitHub, please **make sure your code is not publicly accessible**.

To repeat (because we have taught this class before): make frequent small commits as you work, and use commit messages that identify what changed and why.

具体来说, 我们希望您:

- 使用 <https://en.cppreference.com> 上的语言文档作为参考资源。(我们建议您避免使用 `cplusplus.com`, 该网站内容更容易过时。)
- 切勿使用 `malloc()` 或 `free()`。
- 切勿使用 `new` 或 `delete`。
- 基本上永远不要使用原始指针 (*), 仅在必要时使用“智能”指针 (`unique_ptr` 或 `shared_ptr`)。(在 CS144 课程中你不需要使用这些。)
- 避免使用模板、线程、锁和虚函数。(在 CS144 课程中你不需要使用这些。)
- 避免使用 C 风格字符串 (`char *str`) 或字符串函数 (`strlen()`, `strcpy()`)。这些很容易出错。请改用 `std::string`。
- 永远不要使用 C 风格类型转换 (例如 `(FILE *)x`)。如果必须使用, 请使用 C++ 的 `static_cast` (在 CS144 课程中你通常不需要这样做)。
- 优先通过常量引用传递函数参数 (例如: `const Address & address`)。
- 除非需要修改, 否则每个变量都应声明为 `const`。
- 除非需要修改对象, 否则每个方法都应声明为 `const`。
- 避免使用全局变量, 并尽可能将每个变量的作用域限制在最小范围内。
- 在提交作业前, 请运行 `cmake --build build --target tidy` 获取关于改进 C++ 编程实践相关代码的建议, 并运行 `cmake --build build --target format` 以统一代码格式。

关于使用 Git: 实验以 Git (版本控制) 仓库形式分发—这是一种记录变更、标记版本以辅助调试、追踪源代码来源的方式。请在工作过程中频繁进行小规模提交, 并使用能说明变更内容及原因的提交信息。理想状态下, 每次提交都应能编译通过, 并稳步推进更多测试用例的通过。进行小型的“语义化”提交有助于调试 (若每次提交都能编译通过且信息描述单一明确功能, 调试将更为便捷), 同时通过记录持续进展来防范抄袭指控—这也是任何涉及软件开发职业中的实用技能。评分人员将通过阅读提交信息来了解你完成实验方案的过程。若尚未掌握 Git 使用方法, 请务必在 CS144 答疑时间寻求帮助, 或参考相关教程 (例如 <https://guides.github.com/introduction/git-handbook>)。最后, 虽然我们要求你通过 GitHub 上的私有仓库备份并提交代码, 但请确保你的代码不对外公开。

再次强调 (因为我们之前教过这门课): 在编写代码时, 请频繁进行小规模提交, 并使用能说明更改内容和原因的提交信息。

4.4 Reading the Minnow support code

To support this style of programming, Minnow’s classes wrap operating-system functions (which can be called from C) in “modern” C++. We have provided you with C++ wrappers for concepts we hope you’re broadly familiar with from CS 111, especially sockets and file descriptors.

Please read over the public interfaces (the part that comes after “public:” in the files `util/socket.hh` and `util/file_descriptor.hh`. (Please note that a `Socket` is a type of `FileDescriptor`, and a `TCPSocket` is a type of `Socket`.)

4.5 Writing webget

It’s time to implement `webget`, a program to fetch Web pages over the Internet using the operating system’s TCP support and stream-socket abstraction—just like you did by hand earlier in this lab.

1. From the `build` directory, open the file `../apps/webget.cc` in a text editor or IDE.
2. In the `get_URL` function, implement the simple Web client as described in this file, using the format of an HTTP (Web) request that you used earlier. Use the `TCPSocket` and `Address` classes.
3. Hints:
 - Please note that in HTTP, each line must be ended with “`\r\n`” (it’s not sufficient to use just “`\n`” or `endl`).
 - Don’t forget to include the “Connection: close” line in your client’s request. This tells the server that it shouldn’t wait around for your client to send any more requests after this one. Instead, the server will send one reply and then will immediately end its outgoing bytestream (the one *from* the server’s socket *to* your socket). You’ll discover that your incoming byte stream has ended because your socket will reach “EOF” (end of file) when you have read the entire byte stream coming from the server. That’s how your client will know that the server has finished its reply.
 - Make sure to read and print *all* the output from the server until the socket reaches “EOF” (end of file)—**a single call to read is not enough**.
 - We expect you’ll need to write about eight lines of code.
4. Compile your program by running `cmake --build build`. If you see an error message, you will need to fix it before continuing.
5. Test your program by running `./apps/webget cs144.keithw.org /hello`. How does this compare to what you see when visiting <http://cs144.keithw.org/hello> in a

4.4 阅读 Minnow 支持代码

为了支持这种编程风格，Minnow 的类将操作系统函数（可从 C 语言调用）封装在“现代”C++ 中。我们为你提供了针对一些概念的 C++ 封装，希望你从 CS 111 课程中已对这些概念有大致了解，特别是套接字和文件描述符。

请阅读公共接口（即文件 `util/socket.hh` 和 `util/file_descriptor.hh` 中“public:”之后的部分）。（请注意，`Socket` 是 `FileDescriptor` 的一种类型，而 `TCPSocket` 是 `Socket` 的一种类型。）

4.5 编写 webget

是时候实现 `webget` 了，这是一个利用操作系统 TCP 支持和流套接字抽象来从互联网获取网页的程序——就像你之前在实验中手动操作的那样。

1. 从构建目录中，在文本编辑器或 IDE 中打开文件 `../apps/webget.cc`。
2. 在 `get URL` 函数中，按照本文件中的描述，使用你之前用过的 HTTP (Web) 请求格式，实现简单的 Web 客户端。请使用 `TCPSocket` 和 `Address` 类。
3. 提示：
 - 请注意，在 HTTP 协议中，每一行必须以“`\r\n`”结尾（仅使用“`\n`”或 `endl` 是不够的）。
 - 别忘了在客户端的请求中包含“Connection: close”这一行。这告诉服务器，在此请求之后，不应等待你的客户端发送更多请求。相反，服务器将发送一个回复，然后立即结束其出站字节流（从服务器套接字到你的套接字的字节流）。你会发现，当你的套接字读取完来自服务器的整个字节流时，它会达到“EOF”（文件结束符）。这就是你的客户端如何知道服务器已完成回复的方式。
 - 请确保读取并打印来自服务器的所有输出，直到套接字达到“EOF”（文件结束符）——仅调用一次 `read` 是不够的。
 - 我们预计你需要编写大约八行代码。
4. 通过运行 `cmake --build build` 编译你的程序。如果看到错误信息，你需要先修复它再继续。
5. 通过运行 `./apps/webget cs144.keithw.org /hello` 测试你的程序。这与在浏览器中访问 <http://cs144.keithw.org/hello> 时看到的内容相比如何？

Web browser? How does it compare to the results from Section 3.1? Feel free to experiment—test it with any http URL you like!

6. When it seems to be working properly, run `cmake --build build --target check_webget` to run the automated test. Before implementing the `get_URL` function, you should expect to see the following:

```
$ cmake --build build --target check_webget
Test project /home/cs144/minnow/build
  Start 1: compile with bug-checkers
1/2 Test #1: compile with bug-checkers ..... Passed    1.02 sec
  Start 2: t_webget
2/2 Test #2: t_webget .....***Failed    0.01 sec
DEBUG: Function called: get_URL( "cs144.keithw.org", "/nph-hashier/xyzzy" )
DEBUG: get_URL() function not yet implemented
ERROR: webget returned output that did not match the test's expectations
```

After completing the assignment, you will see:

```
$ cmake --build build --target check_webget
Test project /home/cs144/minnow/build
  Start 1: compile with bug-checkers
1/2 Test #1: compile with bug-checkers ..... Passed    1.09 sec
  Start 2: t_webget
2/2 Test #2: t_webget ..... Passed    0.72 sec

100% tests passed, 0 tests failed out of 2
```

7. The graders will run your `webget` program with a different hostname and path than `make check_webget` runs—so make sure it doesn’t *only* work with the hostname and path used by the unit tests.

5 An in-memory reliable byte stream

By now, you’ve seen how the abstraction of a *reliable byte stream* can be useful in communicating across the Internet, even though the Internet itself only provides the service of “best-effort” (unreliable) datagrams.

To finish off this week’s lab, you will implement, in memory on a single computer, an object that provides this abstraction. (You may have done something similar in CS 110/111.) Bytes are written on the “input” side and can be read, in the same sequence, from the “output” side. The byte stream is finite: the writer can end the input, and then no more bytes can be written. When the reader has read to the end of the stream, it will reach “EOF” (end of file) and no more bytes can be read.

网页浏览器？它与第 3.1 节的结果相比如何？请随意尝试—用你喜欢的任何 http URL 进行测试！

6. 当一切运行正常时，运行 `cmake --build build --target check_webget` 来执行自动化测试。在实现获取 URL 功能之前，你应该会看到以下内容：

```
$ cmake --build build --target check_webget Test
project /home/cs144/minnow/build
Start 1: compile with bug-checkers 1/2 Test #1: compile with bug-
checkers Passed 1.02 sec
开始 2: t_webget 2/2 测试 #2: t_webget ***失败 0.01 秒 调试: 调用了函数: get_URL(
"cs144.keithw.org", "/nph-hashier/xyzzy" ) 调试: get_URL() 函数尚未实现 错误: webget
返回的输出与测试预期不符
```

完成作业后，你将看到：

```
$ cmake --build build --target check_webget 测试
项目 /home/cs144/minnow/build
开始 1: 使用错误检查器编译 1/2 测试 #1: 使用错误检查器编译 通过 1.09 秒
开始 2: t_webget 2/2 测试 #2: t_webget 通过 0.72 秒
```

100% 的测试通过，2 个测试中 0 个失败

7. 评分员将使用与 `make check_webget` 运行时不同的主机名和路径来运行你的 `webget` 程序—因此请确保它不仅适用于单元测试中使用的主机名和路径。

5 一个基于内存的可靠字节流

到目前为止，你已经看到可靠字节流的抽象在互联网通信中是多么有用，尽管互联网本身只提供 keithw 所谓的“尽力而为”（不可靠）数据报服务。

为了完成本周的实验，你将在单台计算机的内存中实现一个提供这种抽象的对象。（你可能在 CS 110/111 课程中做过类似的事情。）字节从“输入”端写入，并可以按相同顺序从“输出”端读取。字节流是有限的：写入者可以结束输入，之后便不能再写入更多字节。当读取者读取到流的末尾时，它将到达“EOF”（文件结束符），之后便不能再读取更多字节。

Your byte stream will also be *flow-controlled* to limit its memory consumption at any given time. The object is initialized with a particular “capacity”: the maximum number of bytes it’s willing to store in its own memory at any given point. The byte stream will limit the writer in how much it can write at any given moment, to make sure that the stream doesn’t exceed its storage capacity. As the reader reads bytes and drains them from the stream, the writer is allowed to write more. Your byte stream is for use in a *single* thread—you don’t have to worry about concurrent writers/readers, locking, or race conditions.

To be clear: the byte stream is finite, but it can be *almost arbitrarily long*⁴ before the writer ends the input and finishes the stream. Your implementation must be able to handle streams that are much longer than the capacity. The capacity limits the number of bytes that are held in memory (written but not yet read) at a given point, but does not limit the length of the stream. An object with a capacity of only one byte could still carry a stream that is terabytes and terabytes long, as long as the writer keeps writing one byte at a time and the reader reads each byte before the writer is allowed to write the next byte.

Here’s what the interface looks like for the writer:

```
void push( std::string data ); // Push data to stream, but only as much as available capacity allows.
void close();                // Signal that the stream has reached its ending. Nothing more will be written.

bool is_closed() const;      // Has the stream been closed?

uint64_t available_capacity() const; // How many bytes can be pushed to the stream right now?
uint64_t bytes_pushed() const;      // Total number of bytes cumulatively pushed to the stream
```

And here is the interface for the reader:

```
std::string_view peek() const; // Peek at the next bytes in the buffer
void pop( uint64_t len );      // Remove `len` bytes from the buffer

bool is_finished() const;     // Is the stream finished (closed and fully popped)?
bool has_error() const;       // Has the stream had an error?

uint64_t bytes_buffered() const; // Number of bytes currently buffered (pushed and not popped)
uint64_t bytes_popped() const;   // Total number of bytes cumulatively popped from stream
```

Please open the `src/byte_stream.hh` and `src/byte_stream.cc` files, and implement an object that provides this interface. As you develop your byte stream implementation, you can run the automated tests with `cmake --build build --target check0`.

If all tests pass, the `check0` test will then run a speed benchmark of your implementation. Anything **faster than 0.1 Gbit/s** (in other words, 100 million bits per second) is acceptable for purposes of this class, for the three pop lengths tested. (It is possible for an implementation to perform faster than 10 Gbit/s, but this depends on the speed of your computer and is not required.)

For any late-breaking questions, please check out the lab FAQ on the [course website](#) or ask your classmates or the teaching staff in the lab session (or on EdStem).

⁴At least up to 2⁶⁴ bytes, which in this class we will regard as essentially arbitrarily long

你的字节流也会受到流量控制，以限制其在任意时刻的内存消耗。该对象在初始化时会设定一个特定的“容量”：即它在任意时刻愿意存储在自己内存中的最大字节数。字节流会限制写入者在任意时刻的写入量，以确保流不会超过其存储容量。当读取者读取字节并将其从流中排出时，写入者才被允许写入更多内容。你的字节流用于单线程环境—无需担心并发写入/读取、锁定或竞态条件。

需要明确的是：字节流是有限的，但在写入者结束输入并完成流之前，它可以几乎任意长。你的实现必须能够处理远大于容量的流。容量限制了在某个时间点内存中保存的字节数（已写入但尚未读取），但并不限制流的长度。一个容量仅为 1 字节的对象仍然可以承载长达数 TB 的流，只要写入者每次写入一个字节，并且读取者在写入者被允许写入下一个字节之前读取每个字节。

以下是写入者的接口：

```
void push( std::string data ); // 将数据推入流，但仅限可用容量允许的数量。
void close();                // 表示流已到达终点。不再写入任何内容。

bool is_closed() const;      // 流是否已关闭？

uint64_t available_capacity() const; // 当前可向流中推送多少字节？
uint64_t bytes_pushed() const;      // 累计推送到流中的总字节数
```

以下是读取器的接口：

```
std::string_view peek() const; // 查看缓冲区中的下一个字节
void pop( uint64_t len );      // 从缓冲区中移除 `len` 个字节

bool is_finished() const;     // 流是否已结束（已关闭且完全弹出）？
bool has_error() const;       // 流是否出现错误？

uint64_t bytes_buffered() const; // 当前缓冲的字节数（已推送但未弹出）
uint64_t bytes_popped() const;   // 从流中累计弹出的总字节数
```

请打开 ``src/byte_stream.hh`` 和 ``src/byte_stream.cc`` 文件，实现一个提供该接口的对象。在开发字节流实现的过程中，你可以使用 ``cmake --build build --target check0`` 命令运行自动化测试。

如果所有测试均通过，``check0`` 测试将运行你的实现的性能基准测试。对于本课程而言，在测试的三种数据包长度下，速度超过 0.1 Gbit/s（即每秒 1 亿比特）即可接受。（实现速度有可能超过 10 Gbit/s，但这取决于你的计算机性能，并非强制要求。）

如有任何最新问题，请查阅课程网站上的实验常见问题解答，或在实验课上向同学或教学人员（或通过 EdStem）提问。

⁴至少支持到 2 字节，在本课程中我们将视其为本质上任意长度。

What's next? Over the next four weeks, you'll implement a system to provide the same interface, no longer in memory, but instead over an unreliable network. This is the Transmission Control Protocol—and its implementations are arguably the **most prevalent computer program in the world**.

6 Submit

- 1. In your submission, please only make changes to `webget.cc` and the source code in the top level of `src` (`byte_stream.hh` and `byte_stream.cc`). Please don't modify any of the tests or the helpers in `util`.
- 2. Remember to make small commits as you code, with good commit messages. After making a commit, back up your VM's repository to your private GitHub repository often by running `git push github`. Your code needs to be committed and pushed to GitHub for it to be gradable.
- 3. Before handing in any assignment, please run these in order:
 - (a) Make sure you have committed all of your changes to the Git repository. You can run `git status` to make sure there are no outstanding changes. Remember: make small commits as you code.
 - (b) `cmake --build build --target format` (to normalize the coding style)
 - (c) `cmake --build build --target check0` (to make sure the automated tests pass)
 - (d) Optional: `cmake --build build --target tidy` (suggests improvements to follow good C++ programming practices)
- 4. Finish editing `writeups/check0.md`, filling in the number of hours this assignment took you and any other comments.
- 5. Make sure your code is committed and pushed to your private GitHub repository (`git push github`).
- 6. There will be a Gradescope assignment due Sunday 11:59 p.m. for you to submit the commit ID of your submission.
- 7. Please let the course staff know ASAP of any problems at the Wednesday lab session, or by posting a question on EdStem. Good luck and welcome to CS144!

接下来呢？在接下来的四周里，你将实现一个系统，提供相同的接口，但不再基于内存，而是通过一个不可靠的网络。这就是传输控制协议——其实现可以说是全球最普遍的计算机程序。

6 提交

- 1. 在提交时，请仅修改 `webget.cc` 以及 `src` 顶层目录中的源代码 (`byte_stream.hh` 和 `byte_stream.cc`)。请不要修改任何测试文件或 `util` 中的辅助程序。
- 2. 请记得在编码过程中进行小规模提交，并附上良好的提交信息。每次提交后，经常通过运行 `git push github` 将虚拟机的仓库备份到你的私有 GitHub 仓库。你的代码需要提交并推送到 GitHub 才能进行评分。
- 3. 提交任何作业前，请按顺序执行以下操作：(a) 确保已将所有更改提交至 Git 仓库。
 - 运行 `git status` 来确认没有未提交的更改。请记住：在编码过程中进行小规模提交。
 - (b) `cmake --build build --target format` (以规范化编码风格)
 - (c) `cmake --build build --target check0` (确保自动化测试通过)
 - (d) 可选: `cmake --build build --target tidy` (提供改进建议以遵循良好的 C++ 编程实践)
- 4. 完成编辑 `writeups/check0.md`，填写完成本次作业所花费的小时数及其他备注。
- 5. 确保你的代码已提交并推送至你的私有 GitHub 仓库 (`git push github`)。
- 6. 本周日晚上 11:59 前，Gradescope 上会有一个作业，你需要提交你提交版本的 commit ID。
- 7. 如有任何问题，请尽快在周三的实验课上告知课程工作人员，或在 EdStem 上提问。祝你好运，欢迎来到 CS144!